UNIT - 3

2 MARKS QUESTIONS

1. Define knowledge based agent.
2. Write structure of generic knowledge-based agent.
3. List the steps of knowledge engineering process.
4. What is inference?
5. Define syntax and symantics in logic with example.

6. Specify environment information of wumpus world problem.

Grid Layout: 4x4 grid, start at [1,1], facing right. Gold, wumpus, and pits randomly placed.
Hazards:20% pit probability per room, wumpus in one room. Death by pit or wumpus.
Goals:Find and pick up gold (+1000 points), avoid pits and wumpus (-1000 points).
Actions:Move forward, turn left/right, grab, shoot arrow (once). Walls restrict movement, bump gives feedback.
Sensory Input Stench (wumpus), breeze (pits), glitter (gold), bump (walls), scream (wumpus killed).

7. What is entailment?

Entailment is the logical relation where a sentence α entails a sentence β if and only if in every model where α is true, β is also true. This means that the truth of βis contained within the truth of α.

8. Define atomic sentence in propositional logic. Give an example.

In propositional logic, an atomic sentence, also known as an atomic proposition, consists of a single proposition symbol which represents a statement that can either be true or false. These symbols are indivisible and do not contain meaningful subcomponents.

For example, the symbol W1,3W_{1,3}W1,3 might represent the proposition "the wumpus is in [1,3]". Here, W1,3W_{1,3}W1,3 is an atomic sentence. The symbols "True" and "False" are special atomic sentences with fixed meanings, always true and always false, respectively.

9. Define complex sentence in propositional logic. Give an example.

A complex sentence in propositional logic is a statement formed by combining simpler statements using logical connectives. These connectives include AND (∧), OR (∨), NOT (¬), and IMPLIES (→). Complex sentences allow us to express relationships between propositions and create more intricate logical structures.

~Rrother(LeftLeg(Richard)

,

John)

Brother(Richard, John) A Brother(John, Richard)

King (Richard) v *King (John)*

7

King (Richard) + *King (John)* .

This example combines two simpler sentences (propositions) using the logical connectives NOT, OR, and AND to form a complex sentence in propositional logic

10. List any four logical connectives used in propositional logic.

Four logical connectives used in propositional logic are:

1. AND (∧) - Represents conjunction.

2. OR (∨) - Represents disjunction.

3. NOT (¬) - Represents negation.

4.IMPLIES (→) - Represents implication.

11. When do we say that two sentences are logically equivalent?

The logical equivalence two sentences a and B are logically equivalent.if they are true in the same set of models.We write this as a=B.for example we can easily show that PAQ and QAP are logically equivalent

12. What is tautology? Give an example.

Valid sentences are also known as tautologies they are necessarily true and hence vacuous. Because the sentence Due is true in all models ,every valid sentence is logically equivalent to Due.

13. What is valid sentence? Give an example.

A sentence is valid if it is true in all models.For example the sentence p or negation p valid.

14. What is satisfiable sentences.? Give an example.

A sentence is satisfiable if it is true in some model.For example ,the knowledge base given earlier,(R1 A R2 A R3 A)is satisfiable because there are three models in which it is true .

15. Differentiate between valid and satisfiable sentences.

Valid sentences:
A sentence is valid if it is true in all models
.valid sentences are also known as tautology
Satisfiable sentence:
A sentence is satisfiable if it is true in some model

16. Write the Modus ponens Rule. Give example.

17. Write And-Elimination rule. Give example.

- And Elimination
- From a conjunction, any of the conjuncts can be inferred

$$\frac{\alpha \wedge \beta}{\alpha}$$

- Example
- (WumpusAhead ∧ WumpusAlive), WumpusAlive can be inferred.

18. Write Biconditional elimination rule?

Inference Rules – Biconditional Elimination

- The equivalence for biconditional elimination yields the two inference rules

$$\frac{\alpha \Leftrightarrow \beta}{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)} \quad \text{and} \quad \frac{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)}{\alpha \Leftrightarrow \beta}$$

19. Define CNF.Give an example.

20. Define sound and complete in propositional logic.

Soundness:

- **Definition**: A logical system (or inference procedure) is said to be sound if every conclusion it reaches is true whenever all of its premises are true.
- **Explanation**: In other words, if a logical system is sound, it guarantees that whenever it deduces something (concludes something as true), that conclusion is indeed true in all possible models where the premises (initial statements or axioms) are also true. Soundness ensures correctness in reasoning; if a system is sound, it doesn't lead to false conclusions from true premises.

Completeness:

- **Definition**: A logical system (or inference procedure) is considered complete if it can derive every statement that is true in all models from its premises.
- **Explanation**: Completeness guarantees that the logical system can find a proof (or derive) every statement that is logically entailed by the premises. In other words, if a system is complete, it ensures that it can find the truth of every statement that is

necessarily true given the initial set of true premises. Completeness is about not missing any valid conclusions that logically follow from the premises.

21. What is ground resolution theorem?

The completeness theorem for resolution in prlopositional logic is called the ground resolution theorem:
If a set of clauses is unsatisfiable, then the resolution closure of those clauses contains the empty clause.
We prove this theorem by demonstrating its contrapolsitive: if the closure RC(S) does not contain the empty clause, then S is satisfiable. In fact, we can construct a model for S with suitable truth values for P1, . . . , Pk. The construction procedure is as follows:
For i from 1 to k,

- If there is a clause in RC(S) containing the literal Pi such that all its other literals are false under the assignment chosen for PI, . . . , Pi-l, then assign false to Pi.
- Otherwise, assign true to Pi.

It remains to show that this assignment to P1, . . . , Pk is a model of S, provided that RC(S) is closed under resolution and does not contain the empty clause.

22. What is a horn clause? Give an example.

Horn clause: A clause which is a disjunction of literals with at most one positive literal is known as horn clause. Hence all the definite clauses are horn clauses.
Example: (¬ p V ¬ q V k). It has only one positive literal k.

23. What is forward chaining in propositional logic?

Forward chaining is a method of reasoning in artificial intelligence in which inference rules are applied to
Exisiting data until an endpoint(goal) is achieved.which starts with atomic sentences in the knowledge base
And applies inference in the forward direction to extract more data until a goal is reached.

24. What are first order logic?

First Order Logic (FOL) is a formal system that extends propositional logic by including quantifiers (∀ for all, ∃ for some) and predicates to express statements about objects and their relationships. It allows for the expression of more complex statements about the structure and properties of objects within a domain.

25. Give example for atomic and complex sentences in FOL.

Atomic
Ex: Grade(Alice,CS101)>80
The grade of Alice in CS101 is greater than 80.

Complex
Ex: Student(Alice)∧Enrolled(Alice,CS101)

Alice is a student and enrolled in CS101.

26. What are Universal quantifiers. Give example.

Universal quantifiers are logical symbols used in formal logic to indicate that a particular property or predicate applies to all elements within a specified domain. The universal quantifier is usually denoted by the symbol "∀" (an upside-down "A"), which can be read as "for all" or "for every."

Example:
Consider the statement:
$\forall x(P(x))$

This statement reads as "For all x, P(x) is true," meaning that the property P holds for every element x in the domain of discourse.

27. What are Existential quantifiers. Give example.

- Existential quantifiers express that the statement within its scope is true for at least one instance of something.
- It is denoted by the logical operator 3, which resembles as inverted E. When it is used with a predicate variable then it is called as an existential quantifier.
- In Existential quantifier we always use AND or Conjunction symbol (A).
- If x is a variable, then existential quantifier will be 3x or 3(x). And it will be read as:
    - ☐ There exists a 'x.'
    - ☐ For some 'x.'
    - ☐ For at least one 'x.'

- Example:
    - ☐ Some boys are intelligent.
    - ☐ ∃x: boys(x) A intelligent(x)

(There are some x where x is a boy who is intelligent)

28. What are nested quantifiers. Give example.

Nested quantifiers are used in mathematics and logic to express statements involving multiple variables over a domain. They involve quantifiers like "for all" (∀) and "there exists" (∃).

Here's an example to illustrate nested quantifiers:

Let P(x,y) represent the statement "x is less than y".

$\forall x \forall y$ P(x,y)

This means "for all x and for all y, x is less than y". In other words, every pair of elements (x, y) from the domain satisfies the condition that x is less than y.

29. Give an example for equality in FOL.

First-order logic includes one more way to make atomic sentences, other than using a predicate and terms as described earlier. We can use the **equality symbol** to make statements to the effect that two terms refer to the same object.

For example,

Father(John) =Henry

30. What are diagnostic rule in FOL. Give example

31. What are casual rules. Give an example.

32. State the role of universal instantiation with an example.

Universal instantiation :

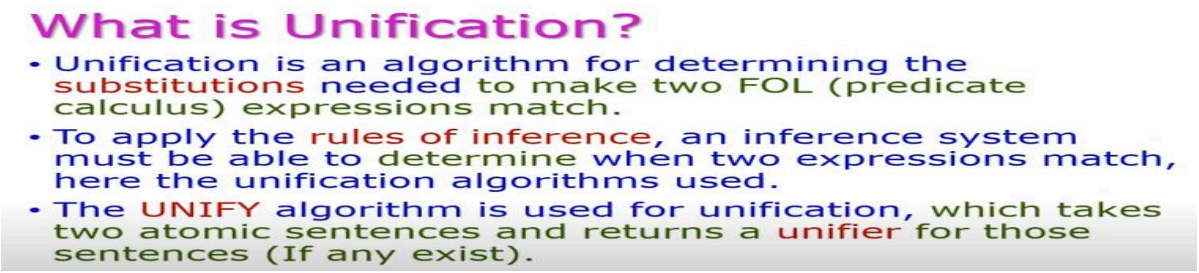
The rule of Universal Instantiation (UI for short) says that we can infer any sentence obtained by substituting a ground term (a term without variables) for the variable. To write out the inference rule formally we use the notion of substitutions.

33. State the role of existential instantiation with an example.

The corresponding Existential Instantiation rule: for the existential quantifier is slightly more complicated. For any sentence a, variable v, and constant symbol k that does not appear elsewhere in the knowledge base,

For example, from the sentence 3 2 Crown(x) A OnHead (x, John) we can infer the sentence Crown(C1) A OnHead(Cl, John) as long as Cl does not appear elsewhere in the knowledge base.

34. What are unification. Give example



- Example:
- Find the MGU for Unify{King(x), King(John)}

- Let α1 = King(x), α2 = King(John),
- Substitution θ = {John/x} is a unifier for these atoms and applying this substitution, and both expressions will be identical.

- We can get the inference immediately if we can find a substitution θ such that *King(x)* and *Greedy(x)* match *King(John)* and *Greedy(y)*

θ = {x/John,y/John} works

- Unify(α,β) = θ if αθ = βθ

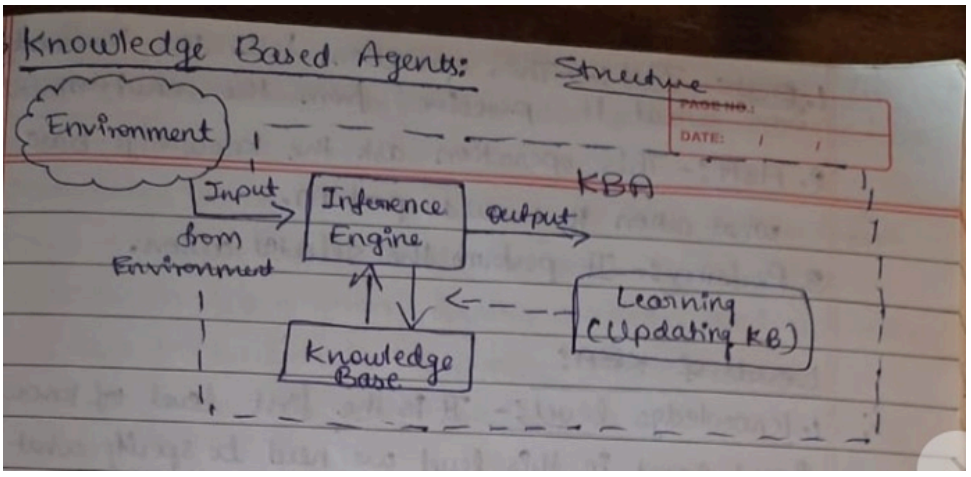
p	q	θ
Knows(John,x)	Knows(John,Jane)	{x/Jane}}
Knows(John,x)	Knows(y,Jack)	{x/Jack,y/John}}
Knows(John,x)	Knows(y,Mother(y))	{y/John,x/Mother(John)}}
Knows(John,x)	Knows(x,Jack)	{fail}

35. What are Lifting. Give example.

Long Answer Questions (3,4,5,6 Marks)

1. Write a note on knowledge based agent.

Knowledge Based Agent is composed Inference Engine (System) an agent should be able to represent states, actions, etc. An agent should be able to take new perceptions. An agent can update internal representation of the world and it can reduce appropriate actions.



The KBA take input from the environment by perceiving the environment. Input is taken by the inference engine of the agent and which also communicate with knowledge base by learning new knowledge. knowledge base is a central component of a knowledge based agent. It is a collection of sentences. These sentences are expressed in a language which is called a knowledge Representation Language. the knowledge base stores facts about the world. Knowledge base is required for updating knowledge as an agent to learn with experiences and actions and take actions as per the knowledge.

Inference means deriving new sentences from old. It allows to add new sentences to the knowledge base.

Operation performed by KBA:

1. Tell: This operation tells the knowledge base what it perceives from the environment.
2. Ask: This operation asks the knowledge base what action it should perform.
3. Perform: It performs the selected action.

Levels of KBA:

1. **Knowledge level**: It's the first level of knowledge-based agent. In this level, we need to specify what the agent knows & what the agent goals are.Example :Suppose a taxi agent needs to go from station A to station B. And he knows the way from A to B, this comes from the knowledge level.

2. **Logical level:** It is except the taxi agent to reach to the destination B.

3. **Implementation level**: It is a physical representation of logic & knowledge. At the implementation level, an agent performs actions as per logical & knowledge level. At this level, the taxi agent actually implements his knowledge of logic, so that he reaches to the destination.

2. Explain PEAS of wumpus world problem.
Performance measure: +I000 for piclung up the gold, -1000 for falling into a pit or being eaten by the wumpus, -1 for each action taken and -10 for using up the arrow.
0 Environment: A 4 x 4 grid of rooms. The agent always starts in the square labeled [1,1], facing to the right. The locations of the gold and the wumpus are chosen randomly, with a uniform distribution, from the squares other than the start square. In addition, each square other than the start can be a pit, wnth probability 0.2. 0 Actuators: The agent can move forward, turn left by 90°, or turn right by 90". The agent dies a miserable death if it enters a square containing a pit or a live wumpus. (It is safe, albeit smelly, to enter a square with a dead wunlpus.) Moving forward has no
3. Explain with the steps to convert given sentence into CNF B1,1↔(P1,2 □ P2,1)

To convert the given sentence $B_{1,1} \leftrightarrow (P_{1,2} \vee P_{2,1})$ into Conjunctive Normal Form (CNF), we need to follow these steps:

1. **Eliminate biconditional ($\leftrightarrow$):**
   The biconditional $A \leftrightarrow B$ is equivalent to $(A \rightarrow B) \wedge (B \rightarrow A)$. Therefore,

   $$B_{1,1} \leftrightarrow (P_{1,2} \vee P_{2,1})$$

   becomes:

   $$(B_{1,1} \rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \rightarrow B_{1,1})$$

2. **Eliminate implications ($\rightarrow$):**
   The implication $A \rightarrow B$ is equivalent to $\neg A \vee B$. Applying this to both implications:

   $$(\neg B_{1,1} \vee (P_{1,2} \vee P_{2,1})) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$$

   The second part can be simplified using De Morgan's laws.

3. **Apply De Morgan's laws:**
   De Morgan's laws state $\neg(A \vee B) = \neg A \wedge \neg B$. Applying this to $\neg(P_{1,2} \vee P_{2,1})$:

   $$(\neg B_{1,1} \vee (P_{1,2} \vee P_{2,1})) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1})$$

4. **Distribute $\vee$ over $\wedge$ to get CNF:**

   - The first part $(\neg B_{1,1} \vee (P_{1,2} \vee P_{2,1}))$ is already in a disjunction of literals, so it is in CNF.
   - For the second part $(\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1}$, distribute $\vee$ over $\wedge$:

     $$(\neg P_{1.2} \vee B_{1.1}) \wedge (\neg P_{2.1} \vee B_{1.1})$$

Putting it all together:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$$

Therefore, the given sentence $B_{1,1} \leftrightarrow (P_{1,2} \vee P_{2,1})$ in Conjunctive Normal Form (CNF) is:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$$

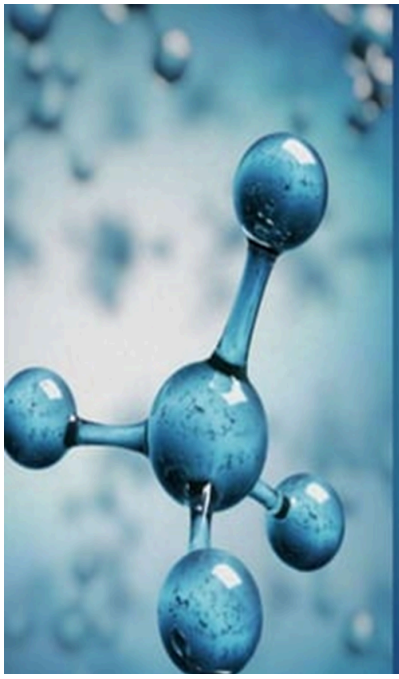
4. Briefly explain atomic and complex sentences in propositional logic with suitable examples.

Propositional Logic

- The Syntax of propositional logic defines the allowable sentences.
- The atomic sentences consist of a single proposition symbol Each such symbol stands for a proposition that can be true or false.
- Example: Sunday is Holiday. (true proposition)
  2+1=4 (False proposition)

some boys like to play cricket.(Not a propositional logic)

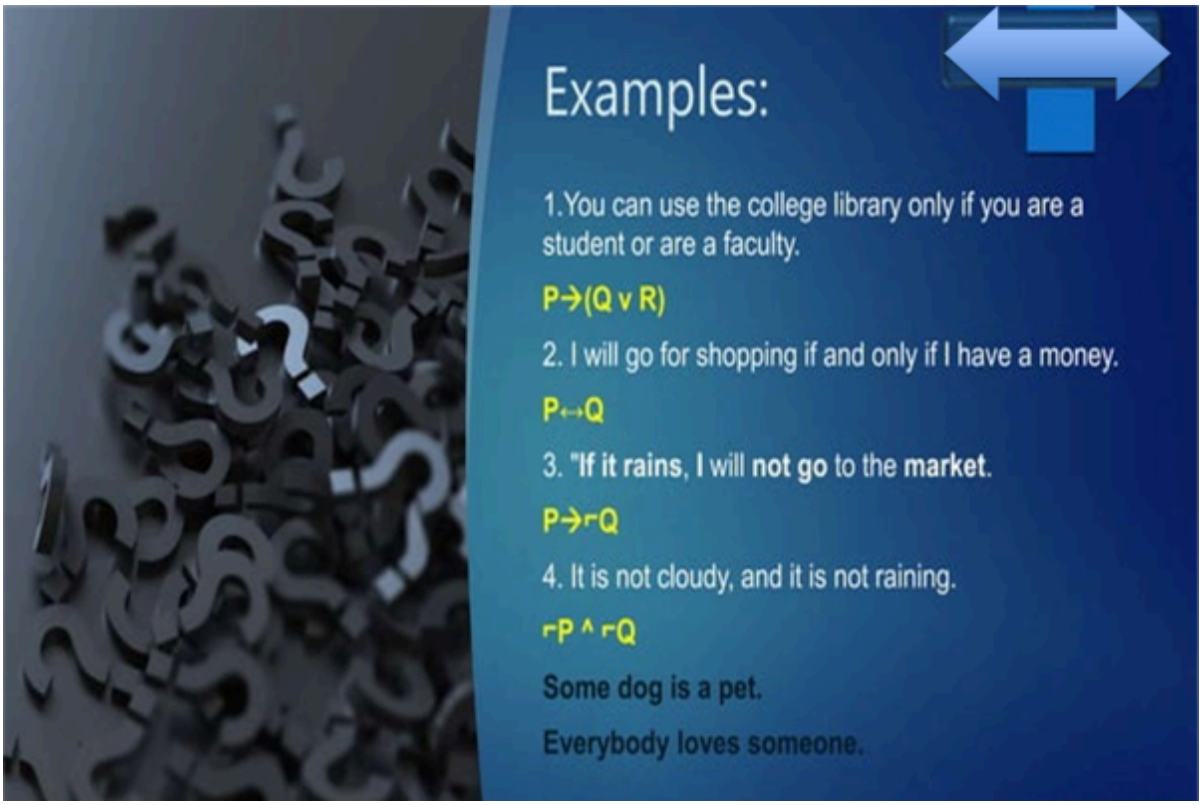
- Complex sentences are constructed from simpler sentences, using parentheses and logical connective.



Connectives

OMega TechEd

Symbol	Name	Meaning
¬	(Not) Negation	If P is true , ¬P will be false and vice versa.
^	(and) Conjunction	(P^Q) is true if both P and Q are true otherwise false.
v	(or) Disjunction	(P v Q) is true if either P or Q is true (or both) otherwise false.
→	implies	If P happens then Q happens.
↔	Double implication	P happens if and only if Q happens.



5. Consider KB=R1 □ R2 R1=A □ C R2=B □ □C α=A □ B Then verify KB ⊨ α using propositional logic.

Consider [KB = R1 \land R2] [R1 = A \lor C] [R2 = B \lor \neg C] [\alpha = A \lor B]

Then verify (KB \models \alpha) using propositional logic.

"Let's solve this step-by-step using propositional logic.
Given:(KB = R1 \land R2)(R1 = A \lor C)(R2 = B \lor \neg C)(\alpha = A \lor B)

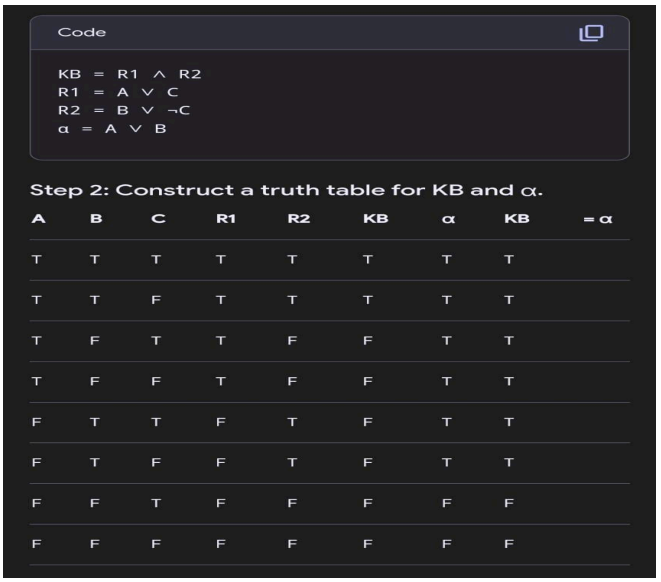
To Prove:[KB \models \alpha]

Proof:Using resolution:Express (KB) in Conjunctive Normal Form (CNF):
[KB = (A \lor C) \land (B \lor \neg C)]Negate (\alpha): [\neg \alpha = \neg (A \lor B) = \neg A \land \neg B]Add the negated (\alpha) to (KB): [(A \lor C) \land (B \lor \neg C) \land (\neg A) \land (\neg B)]

Apply the resolution rule to derive a contradiction:From ((A \lor C)) and (\neg A), we get (C).From ((B \lor \neg C)) and (\neg B), we get (\neg C).

Now we have (C) and (\neg C), which is a contradiction.

Since we derived a contradiction, it means that (KB \models \alpha).Therefore, the knowledge base (KB) entails (\alpha).



6. Consider given KB and using inference rule make the conclusion neither [1,2] nor [2,1] contains pit.

The KB consists of the following statements:

1. $R1 : \neg P_{1,1}$ (There is no pit in $[1, 1]$)
2. $R2 : B_{1,1} \leftrightarrow (P_{1,2} \vee P_{2,1})$ (The breeze in $[1, 1]$ is equivalent to the existence of a pit in $[1, 2]$ or $[2, 1]$)
3. $R3 : B_{2,1} \leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$ (The breeze in $[2, 1]$ is equivalent to the existence of a pit in $[1, 1]$, $[2, 2]$, or $[3, 1]$)
4. $R4 : \neg B_{1,1}$ (There is no breeze in $[1, 1]$)
5. $R5 : B_{2,1}$ (There is a breeze in $[2, 1]$)

We need to conclude that neither $[1, 2]$ nor $[2, 1]$ contains a pit.

Step-by-Step Inference:

1. From $R4$: $\neg B_{1,1}$
   Since there is no breeze in $[1, 1]$.

2. Use $R2$ with $\neg B_{1,1}$:
   $B_{1,1} \leftrightarrow (P_{1,2} \vee P_{2,1})$

   Since $\neg B_{1,1}$ is true, this implies:

   $$\neg(P_{1,2} \vee P_{2,1})$$

   Which simplifies to:

   $$\neg P_{1,2} \wedge \neg P_{2,1}$$

Thus, we can conclude:

$$\neg P_{1,2} \text{ and } \neg P_{2,1}$$

This means that there are no pits in $[1, 2]$ and $[2, 1]$.

7. Decide whether each of the following sentences is valid, unsatisfiable, or neither using truth tables or the equivalence rules.

8. Explain the various logical connectives used in propositional logic.

1. **¬ (Not) Negation:**
   - **Symbol:** ¬
   - **Meaning:** Negation inverts the truth value of a given proposition. If a proposition P is true, then its negation $\neg P$ will be false, and if P is false, then $\neg P$ will be true. This connective is crucial for expressing the opposite of a given statement.
2. **∧ (And) Conjunction:**
   - **Symbol:** ∧
   - **Meaning:** Conjunction is used to combine two propositions and is true only if both propositions are true. For $P \wedge Q$ (P and Q), the statement is true if both P and Q are true; otherwise, it is false. This connective is fundamental for expressing conditions that require multiple criteria to be met simultaneously.
3. **∨ (Or) Disjunction:**
   - **Symbol:** ∨
   - **Meaning:** Disjunction represents an inclusive "or" and is true if at least one of the propositions is true. For $P \vee Q$ (P or Q), the statement is true if P is true, Q is true, or both are true. It is only false when both P and Q are false. This connective is useful for scenarios where multiple alternatives are acceptable.
4. **→ (Implies):**
   - **Symbol:** →
   - **Meaning:** Implication describes a conditional relationship between two propositions. For $P \rightarrow Q$ (If P then Q), the statement is true in all cases except when P is true and Q is false. This means that if P happens, then Q must also happen; otherwise, the implication holds true. This connective is essential for expressing dependencies and causations.

5. **↔ (Double implication):**
   - **Symbol:** ↔
   - **Meaning:** Double implication, or bi-conditional, indicates that both propositions must have the same truth value for the statement to be true. For $P \leftrightarrow Q$ (P if and only if Q), the statement is true if both P and Q are either true or false. This connective is used to express equivalence between propositions, where one is true if and only if the other is true.

9. Explain entailment in wumpus world problem with example

Entailment

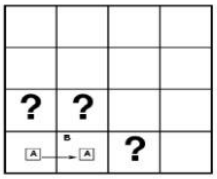
- Entailment means that one thing follows from another:

$$KB \models \alpha$$

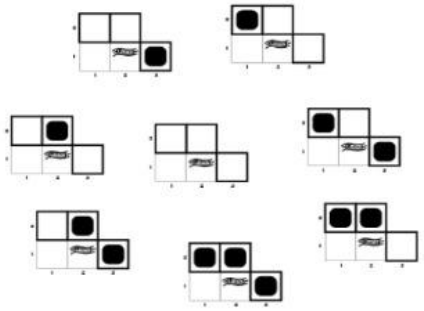
- Knowledge base *KB* entails sentence α if and only if α is true in all worlds where *KB* is true
  - If α true then *KB* must also be true
  - E.g., the KB containing "the Giants won" and "the Reds won" entails "Either the Giants won or the Reds won"
  - E.g., x+y = 4 entails 4 = x+y
  - Entailment is a relationship between sentences (i.e., syntax) that is based on semantics

Entailment in the wumpus world

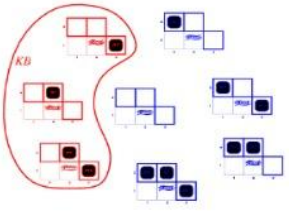
- Situation after detecting nothing in [1,1], moving right, breeze in [2,1]



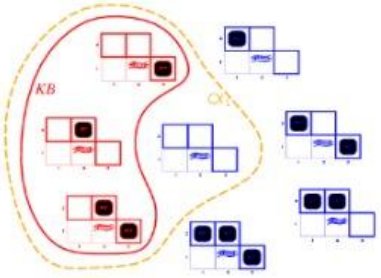
3 Boolean choices ⇒ 8 possible models
for the adjacent squares [1,2], [2,2] and [3,1] to contain pits



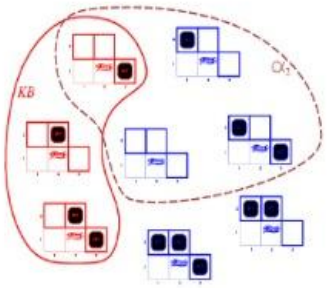
Consider possible models for *KB* assuming only pits



- *KB* = wumpus-world rules + observations
- KB is false in any model in which [1,2] contains a pit, because there is no breeze in [1,1]



- Consider α_1 = "[1,2] is safe" = "There is no pit in [1,2]"
- *In every model KB is true* α_1 *is also true*
- *KB* $\models \alpha_1$, proved by model checking
- We can conclude that there is no pit in [1,2]



- Consider α_2 = "[2,2] is safe" = "There is no pit in [2,2]"
- *In some models in which KB is true* α_2 *is false*
- *KB* $\not\models \alpha_2$
- We cannot conclude that there is no pit in [2,2]

10.Explain wumpus world game with diagram and agent program of it.

11.Write a note on reasoning patterns in propositional logic.

This section covers standard patterns of inference that can be applied to derive chains of conclusions that lead to the desired goal. These patterns of inference are called **inference rules**. The best-known rule is called **Modus Ponens** and is written as follows:

$$\frac{\alpha \Rightarrow \beta, \quad \alpha}{\beta}$$

The notation means that, whenever any sentences of the form a =>β and a are given, then the sentence β

can be inferred. For example, if (WumpusAhead A WumpusAlive) => Shoot and (WumpusAhead A WumpusAlive) are given, then Shoot can be inferred.

AND.ELIMINATION Another useful inference rule is **And-Elimination**, which says that, from a conjunction, any of the conjuncts can be inferred:

$$\frac{\alpha \wedge \beta}{a}$$

For example, from (WumpusAhead A WumpusAlive), WumpusAlive can be inferred. By considering the possible truth values of a and β, one can show easily that Modus Ponens and And-Elimination are sound once and for all. These rules can then be used in any particular instances where they apply, generating sound inferences without the need for enumerating models.

All of the logical equivalences in Figure 7.1 1 can be used as inference rules. For example, the equivalence for biconditional elimination yields the two inference rules:

$$\frac{\alpha \Leftrightarrow \beta}{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)} \quad \text{and} \quad \frac{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)}{\alpha \Leftrightarrow \beta}$$

Not all inference rules work in both directions like this. For example, we cannot run Modus Ponens in the opposite direction to obtain a => β and a from β. Let us see how these inference rules and equivalences can be used in the wumpus world. We start with the knowledge base containing Rl through R5, and show how to prove P_{12}, that is, there is no pit in [1,2]. First, we apply biconditional elimination to R2 to obtain

$$R_6 : \quad (B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$$

Then we apply And-Elimination to R_6 to obtain

$$R_7 : \quad ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1}) .$$

Logical equivalence for contrapositives gives

$$R_8 : \quad (\neg B_{1,1} \Rightarrow \neg(P_{1,2} \vee P_{2,1})) .$$

Now we can apply Modus Ponens with R_8 and the percept R_4 (i.e., $\neg B_{1,1}$), to obtain

$$R_9 : \quad \neg(P_{1,2} \vee P_{2,1}) .$$

Finally, we apply De Morgan's rule, giving the conclusion

$$R_{10} : \quad \neg P_{1,2} \wedge \neg P_{2,1} .$$

That is, neither [1,2] nor [2,1] contains a pit.

The preceding derivation-a sequence of applications of inference rules-is called a **proof**. Finding proofs is exactly like finding solutions to search problems. In fact, if the successor function is defined to generate all possible applications of inference rules, then all of the search algorithms in Chapters 3 and 4 can be applied to find proofs. Thus, searching for proofs is an alternative to enumerating models. The search can go forward from the initial knowledge base, applying inference rules to derive the goal sentence, or it can go backward from the goal sentence, trying to find a chain of inference rules leading from the initial knowledge base. Later in this section, we will see two families of algorithms that use these techniques.

The fact that inference in propositional logic is NP-complete suggests that, in the worst case, searching for proofs is going to be no more efficient than enumerating models. In many practical cases, however, finding a proof can be highly efficient simply because it can ignore irrelevant propositions, no matter how many of them there are. For example, the proof given earlier leading to A 1P2,1 does not mention the propositions B2,1, P2,2, or P3,1. They can be

ignored because the goal proposition, appears only in sentence R4; the other propositions in R4 appear only in R4 and R2; so R1, R3, and R5 have no bearing on the proof. The same would hold even if we added a million more sentences to the knowledge base; the simple truth-table algorithm, on the other hand, would be overwhelmed by the exponential explosion of models.
This property of logical systems actually follows from a much more fundamental property called **monotonicity**. Monotonicity says that the set of entailed sentences can only in- Section 7.5. Reasoning Patterns in Propositional Logic 213 crease as information is added to the knowledge base." For any sentences a and β,

$$\text{if} \quad KB \models \alpha \quad \text{then} \quad KB \wedge \beta \models \alpha \,.$$

For example, suppose the knowledge base contains the additional assertion P stating that there are exactly eight pits in the world. This knowledge might help the agent draw additional conclusions, but it cannot invalidate any conclusion a already inferred-such as the conclusion that there is no pit in [1,2]. Monotonicity means that inference rules can be applied whenever suitable premises are found in the knowledge base-the conclusion of the rule must follow regardless of what else is in the **knowledge base**.
12.Explain BNF (Backus-Naur Form) grammar of sentences in propositional logic along with operator precedence.

BNF (Backus-Naur Form) Grammar for Propositional Logic

Backus-Naur Form (BNF) is a formal notation for specifying the syntax of languages, including programming languages and formal languages used in logic. Here, I'll provide a BNF grammar for propositional logic sentences and discuss the operator precedence.

Propositional Logic Grammar in BNF

In propositional logic, we work with propositions (or statements) that can be true or false, logical operators, and parentheses for grouping.

Let's define the BNF grammar for propositional logic:

<sentence> ::= <atomic_sentence>

| <negation>

| <binary_sentence>

| '(' <sentence> ')'

<atomic_sentence> ::= 'P' | 'Q' | 'R' | ... (propositional variables)

<negation> ::= '¬' <sentence>

<binary_sentence> ::= <sentence> <binary_operator> <sentence>

<binary_operator> ::= '∧' | '∨' | '→' | '↔'

In this grammar:

- `<sentence>` represents any valid sentence in propositional logic.
- `<atomic_sentence>` represents atomic propositions (propositional variables such as P, Q, R, etc.).
- `<negation>` represents the negation of a sentence.
- `<binary_sentence>` represents a sentence formed by combining two sentences with a binary operator.
- `<binary_operator>` represents logical connectives: AND (∧), OR (∨), implication (→), and biconditional (↔).
- Parentheses can be used to group sentences to indicate precedence explicitly.

Operator Precedence

Operator precedence defines the order in which operations are performed in the absence of parentheses. In propositional logic, the typical precedence from highest to lowest is as follows:

1. **Negation (¬)**: Negation has the highest precedence.
2. **Conjunction (∧)**: AND is evaluated next.
3. **Disjunction (∨)**: OR is evaluated after AND.
4. **Implication (→)**: Implication is evaluated after OR.
5. **Biconditional (↔)**: Biconditional has the lowest precedence.

Example

Let's consider the sentence: ¬P ∨ Q ∧ R → S.

Without parentheses, we need to apply the operator precedence rules:

1. **Negation**: Apply ¬ to P first: ¬P
2. **Conjunction**: Apply ∧ to Q and R next: Q ∧ R
3. **Disjunction**: Apply ∨ between ¬P and (Q ∧ R): ¬P ∨ (Q ∧ R)
4. **Implication**: Apply → between (¬P ∨ (Q ∧ R)) and S: (¬P ∨ (Q ∧ R)) → S

This would be represented in the BNF form as:

<sentence> ::= '(' <negation> <disjunction> <conjunction> ')' <implication> <atomic_sentence>

13.Write a short note on standard logical equivalences for arbitrary sentences of propositional logic.

Standard logical equivalences. The symbols a, P, and y stand for arbitrary

sentences of propositional logic.

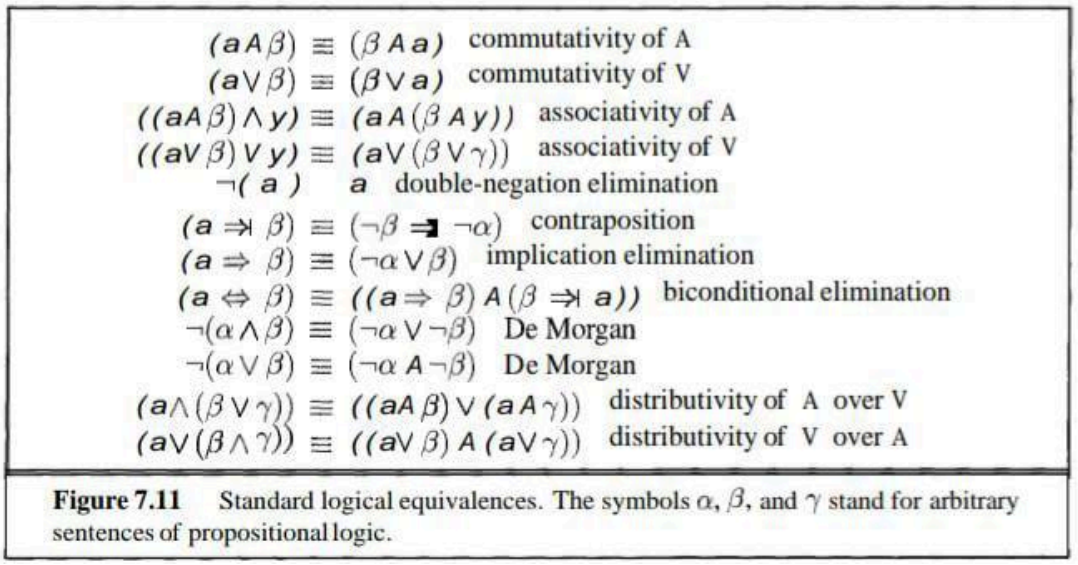


Figure 7.11 Standard logical equivalences. The symbols α, β, and γ stand for arbitrary sentences of propositional logic.

logical equivalence: two sentences a and /3 are logically equivalent
if they are true in the same set of models. We write this as a r P. For example, we can easily show (using truth tables) that P A Q and Q A P are logically equivalent; other equivalences are shown in Figure 7.11. They play much the same role in logic as arithmetic identities do in ordinary mathematics. An alternative definition of equivalence is as follows: for any two sentences a and p,
a r ,b' if and only if a + ,8 and P a

14.Explain the truth table enumeration algorithm for deciding propositional entailment.

The truth table enumeration algorithm is a method for determining whether a knowledge base (KB) entails a proposition (α) in propositional logic. It works by systematically evaluating all possible truth assignments to the propositional symbols and checking if a specific condition holds.

- **Key Concepts**:

1) Knowledge Base (KB): A set of propositions representing background knowledge.
2) Proposition (α): The statement we want to see if the knowledge base entails.
3) Entailment (KB |= α): KB entails α if whenever KB is true, α must also be true.

- **The Algorithm**:

1) Identify Symbols :
   Find all unique propositional symbols used in KB and α.
2) Enumerate Truth Assignments :

Systematically iterate through all possible combinations of truth values (True/False) for each symbol.

3) Evaluate KB :
   For each truth assignment, use a function PL-True?(KB, model) to check if the knowledge base (KB) is true under that specific assignment (model). This function typically involves evaluating the truth value of each proposition in KB based on the assigned truth values to its symbols.
4) Evaluate α :
   If KB is true for the current assignment, evaluate α using the same assignment and a function PL-True?(α, model).
5) Check for Counter-example :
   If KB is true and α is false for a specific assignment, then KB does not entail α. We have found a counter-example where the knowledge base is true but the proposition is false. In this case, the algorithm stops and returns False.
6) Entailment Confirmed :
   If the algorithm iterates through all possible assignments without finding a counter-example (i.e., KB is always true whenever α is also true), then KB entails α. The algorithm returns True.

- **Why it works**:

This algorithm checks entailment by considering every possible scenario (truth assignment). If there exists a scenario where the knowledge base is true but the proposition is false, then entailment doesn't hold.
By checking all scenarios and not finding such a case, the algorithm confirms that whenever the knowledge base is true, the proposition must also be true.

- **Limitations**:

This approach can be computationally expensive for knowledge bases with many symbols. The number of truth assignments grows exponentially with the number of symbols.

15.Represent the following sentences in first - order logic, using a consistent vocabulary

i. Some students took French in spring 2001
ii. Every student who takes French passes it.
iii. Only one student took Greek in spring 2001.
iv. The best score in Greek is always higher than the best score in French.

Let's assume the following:

- Universe of discourse: Students, courses (French, Greek), semesters (Spring 2001)
- Constants: french, greek, spring2001
- Predicates:
  - Student(x): x is a student

- Took(x, y, z): student x took course y in semester z
- Passes(x, y): student x passes course y
- BestScore(x, y): x is the best score in course y

Now, we can represent the given sentences as follows:

i. Some students took French in spring 2001 : There exists an x, such that x is a Student and x Took the course french in the semester spring 2001.

$\exists x$ (Student(x) $\wedge$ Took(x, french, spring2001))

ii. Every student who takes French passes it : For all x, if x is a Student and there exists some semester z where x Took the course french, then x Passes the course french.

$\forall x$ (Student(x) $\wedge$ ($\exists z$ Took(x, french, z)) $\rightarrow$ Passes(x, french))

iii. Only one student took Greek in spring 2001 : There exists an x, such that x is a Student and x Took the course greek in the semester spring2001, and for all y, if y is a Student and y Took the course greek in spring2001, then y is the same as x.

$\exists x$ (Student(x) $\wedge$ Took(x, greek, spring2001) $\wedge$ $\forall y$ ((Student(y) $\wedge$ Took(y, greek, spring2001)) $\rightarrow$ y = x))

iv. The best score in Greek is always higher than the best score in French : For all x and y, if x is the BestScore in the course greek and y is the BestScore in the course french, then x is greater than y.

$\forall x$ $\forall y$ (BestScore(x, greek) $\wedge$ BestScore(y, french) $\rightarrow$ x > y)

16.Write and explain the resolution algorithm for propositional logic with example.

Inference procedures based on resolution work by using the principle of proof by contradiction discussed at the end of Section 7.4. That is, to show that KB + a, we show that (KB A la) is unsatisfiable. We do this by proving a contradiction. A resolution algorithm is shown in Figure 7.12. First, (KB A la) is converted into CNF. Then, the resolution rule is applied to the resulting clauses. Each pair that contains complementary literals is resolved to produce a new clause, which is added to the set if it is not already present. The process continues until one of two things happens:

```
function PL-RESOLUTION(KB, α) returns true or false
  inputs: KB, the knowledge base, a sentence in propositional logic
          α, the query, a sentence in propositional logic

  clauses ← the set of clauses in the CNF representation of KB ∧ ¬α
  new ← { }
  loop do
    for each C_i, C_j in clauses do
      resolvents ← PL-RESOLVE(C_i, C_j)
      if resolvents contains the empty clause then return true
      new ← new ∪ resolvents
    if new ⊆ clauses then return false
    clauses ← clauses ∪ new
```

Figure 7.12 A simple resolution algorithm for propositional logic. The function PL-RESOLVE returns the set of all possible clauses obtained by resolving its two inputs.

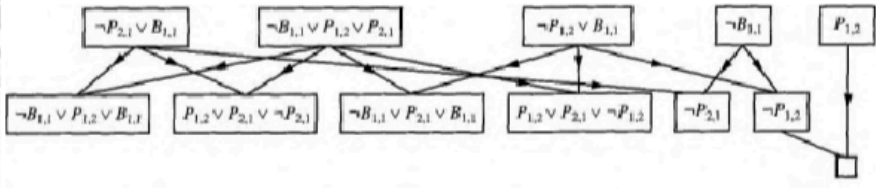


Figure 7.13 Partial application of PL-RESOLUTION to a simple inference in the wumpus world. $\neg P_{1,2}$ is shown to follow from the first four clauses in the top row.

- there are no new clauses that can be added, in which case KB does not entail a; or,
- two clauses resolve to yield the empty clause, in which case KB entails a.

The empty clause-a disjunction of no disjuncts-is equivalent to False because a disjunction is true only if at least one of its disjuncts is true. Another way to see that an empty clause represents a contradiction is to observe that it arises only from resolving two complementary unit clauses such as P and 1P.

We can apply the resolution procedure to a very simple inference in the wumpus world. When the agent is in [l ,I], there is no breeze, so there can be no pits in neighboring squares. The relevant knowledge base is

$$KB = R_2 \wedge R_4 = (B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})) \wedge \neg B_{1,1}$$

we wish to prove α which is, say, $\neg P_{1,2}$. When we c

and we wish to prove a which is, say, YP~,~. When we convert (KB A la) into CNF, we obtain the clauses shown at the top of Figure 7.13. The second row of the figure shows all the clauses obtained by resolving pairs in the first row. Then, when is resolved with --IP~,~, we obtain the empty clause, shown as a small square. Inspection of Figure 7.13 reveals that

many resolution steps are pointless. For example, the clause Bill V 1Blj1 Vis equivalent to True V P1,2 which is equivalent to True. Deducing that True is true is not very helpful. Therefore, any clause in which two complementary literals appear can be discarded.

17.Explain conversion to conjunctive normal form in propositional logic with an example.

- The resolution rule applies only to disjunctions of literals, so it would seem to be relevant only to knowledge bases and queries consisting of such disjunctions.
- How, then, can it lead to a complete inference procedure for all of propositional logic? The answer is that every sentence of propositional logic is logically equivalent to a conjunction of disjunctions of literals. A sentence expressed as a conjunction of disjunctions of literals is said to be in conjunctive normal form or CNF.
- We will also find it useful later to consider the restricted family of K-CNF k-CNF sentences. A sentence in k-CNF has exactly k literals per clause: (el,l v.. . V!l,h) A.. . A v.. . Vtn,h) .
- It turns out that every sentence can be transformed into a 3-CNF sentence that has an equivalent set of models.

Rather than prove these assertions (see Exercise 7.10), we describe a simple conversion procedure. We illustrate the procedure by converting R_2, the sentence $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$, into *CNF*. The steps are as follows:

1. Eliminate $\Leftrightarrow$, replacing $\alpha \Leftrightarrow \beta$ with $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$.
$$(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1}) .$$
2. Eliminate $\Rightarrow$, replacing $\alpha \Rightarrow \beta$ with $\neg\alpha \vee \beta$:
$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1}) .$$
3. CNF requires $\neg$ to appear only in literals, so we "move $\neg$ inwards" by repeated application of the following equivalences from Figure 7.11:
$$\neg(\neg\alpha) \equiv \alpha \quad \text{(double-negation elimination)}$$
$$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta) \quad \text{(De Morgan)}$$
$$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta) \quad \text{(De Morgan)}$$
In the example, we require just one application of the last rule:
$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1}) .$$
4. Now we have a sentence containing nested $\wedge$ and $\vee$ operators applied to literals. We apply the distributivity law from Figure 7.11, distributing $\vee$ over $\wedge$ wherever possible.
$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1}) .$$

- Rather than prove these assertions (see Exercise 7. lo), we describe a simple conversion procedure.

  The original sentence is now in CNF, as a conjunction of three clauses. It is much harder to read, but it can be used as input to a resolution procedure.

18.Explain Forward-chaining algorithm with example in propositional logic.

The best way to understand the algorithm is through an example and a picture. Figure 7.15(a) shows a simple knowledge base of Horn clauses with A and B as known facts. ANWR GRAPH Figure 7.15(b) shows the same knowledge base drawn as an AND-OR graph. In AND-OR graphs, multiple links joined by an arc indicate a conjunction--every link must be

provedwhile multiple links without an arc indicate a disjunction-any link can be proved. It is easy to see how forward chaining works in the graph. The known leaves (here, A and B) are set, and inference propagates up the graph as far as possible. Wherever a conjunction appears, the propagation waits until all the conjuncts are known before proceeding. The reader is encouraged to work through the example in detail.

```
function PL-FC-ENTAILS?(KB, q) returns true or false
  inputs: KB, the knowledge base, a set of propositional Horn clauses
          q, the query, a proposition symbol
  local variables: count, a table, indexed by clause, initially the number of premises
                   inferred, a table, indexed by symbol, each entry initially false
                   agenda, a list of symbols, initially the symbols known to be true in KB

  while agenda is not empty do
    p ← POP(agenda)
    if p = q then return true
    unless inferred[p] do
      inferred[p] ← true
      for each Horn clause c in whose premise p appears do
        decrement count[c]
        if count[c] = 0 then
          PUSH(HEAD[c], agenda)
  return false
```

Figure 7.14 The forward-chaining algorithm for propositional logic. The *agenda* keeps track of symbols known to be true but not yet "processed." The *count* table keeps track of how many premises of each implication are as yet unknown. Whenever a new symbol p from the agenda is processed, the count is reduced by one for each implication in whose premise *p* appears. (These can be identified in constant time if *KB* is indexed appropriately.) If a count reaches zero, all the premises of the implication are known so its conclusion can be added to the agenda. Finally, we need to keep track of which symbols have been processed; an inferred symbol need not be added to the agenda if it has been processed previously. This avoids redundant work; it also prevents infinite loops that could be caused by implications such as $P \Rightarrow Q$ and $Q \Rightarrow P$.

It is easy to see that forward chaining is sound: every inference is essentially an application of Modus Ponens. Forward chaining is also complete: every entailed atomic sentence will be derived. The easiest way to see this is to consider the final state of the inferred table(after the algorithm reaches a fixed point where no new inferences are possible). The table contains true for each symbol inferred during the process, and false for all other symbols. We can view the table as a logical model; moreover, every deiinite clause in the original KB is true in this model. To see this, assume the opposite, namely that some clause a1 A. . .Ask + b is false in the model. Then a1 A . . . A ak must be true in the model and b must be false in the model. But this contradicts our assumption that the algorithm has reached a fi xed point! We can conclude, therefore, that the set of atomic sentences inferred at the fi xed point defi nes a model of the original KB. Furthermore, any atomic sentence q that is entailed by the KB must be true in all its models and in this model in particular. Hence, every entailed sentence q must be inferred by the algorithm.

Forward chaining is an example of the general concept of data-driven reasoning-that is, reasoning in which the focus of attention starts with the known data. It can be used within an agent to derive conclusions from incoming percepts, often without a specific query in mind. For example, the wumpus agent might TELL its percepts to the knowledge base using an incremental forward-chaining algorithm in which new facts can be added to the agenda to - - initiate new inferences. In humans, a certain amount of data-driven reasoning occurs as new information arrives. For example, if I am indoors and hear rain starting to fall, it might occur

to me that the picnic will be canceled. Yet it will probably not occur to me that the seventeenth petal on the largest rose in my neighbor's garden will get wet; humans keep forward chaining under careful control, lest they be swamped with irrelevant consequences.

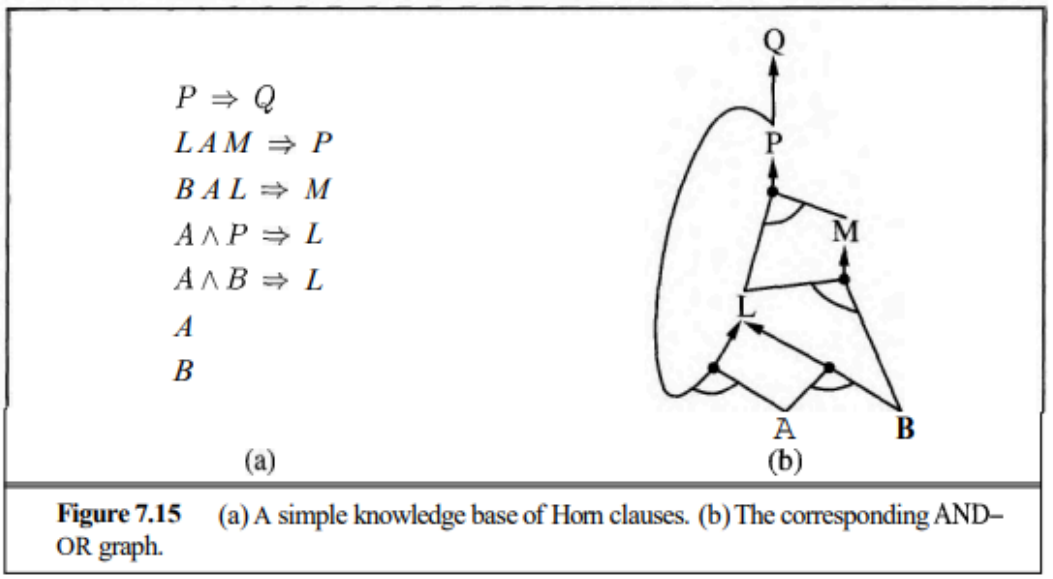


Figure 7.15 (a) A simple knowledge base of Horn clauses. (b) The corresponding AND–OR graph.

19.Write a note on Backtracking Algorithm / DPLL algorithm.

Define backward chaining algorithm.

- Backward-chaining is also known as a backward deduction or backward reasoning method when using an inference engine.
- A backward chaining algorithm is a form of reasoning, which starts with the goal and works backward, chaining through rules to find known facts that support the goal.
- It is known as a top-down approach.
- Backward-chaining is based on modus ponens inference rule.
- In backward chaining, the goal is broken into sub-goal or sub-goals to prove the facts true.
- It is also called as goal-driven approach, as a list of goals decides which rules are selected and used.

20.Explain WALKSAT algorithm in detail.

21.Explain finding pits and wumpuses using logical inference.

Finding pits and wumpuses using logical inference :

Finding pits and wumpuses using logical inference Let us begin with an agent that reasons logically about the location of pits, wumpuses, and safe squares. It begins with a knowledge base that states the "physics" of the wumpus world.

It knows that [1,1] does not contain a pit or a wumpus; that is, lPl,l and square [x, y], it knows a sentence stating how a breeze arises:

$$B_{x,y} \Leftrightarrow (P_{x,y+1} \vee P_{x,y-1} \vee P_{x+1,y} \vee P_{x-1,y}) . \quad (7.1)$$

For every square [x,y], it knows a sentence stating hovv a stench arises:

$$S_{x,y} \Leftrightarrow (W_{x,y+1} \vee W_{x,y-1} \vee W_{x+1,y} \vee W_{x-1,y}) . \quad (7.2)$$

Finally, it knows that there is exactly one wumpus. This is expressed in two parts. First, we have to say that there is *at least one* wumpus:

$$W_{1,1} \vee W_{1,2} \vee \cdot \cdot \cdot \vee W_{4,3} \vee W_{4,4} .$$

Then, we have to say that there is at most one wumpus. One way to do this is to say that for any two squares, one of them must be wumpus-free. With n squares, we get n(n - 1)/2

```
function PL-WUMPUS-AGENT(percept) returns an action
  inputs: percept, a list, [stench,breeze,glitter]
  static: KB, a knowledge base, initially containing the "physics" of the wumpus world
          x, y, orientation, the agent's position (initially 1,1) and orientation (initially right)
          visited, an array indicating which squares have been visited, initially false
          action, the agent's most recent action, initially null
          plan, an action sequence, initially empty

  update x,y,orientation, visited based on action
  if stench then TELL(KB, S,,) else TELL(KB, ¬ S_{x,y})
  if breeze then TELL(KB, B_{x,y}) else TELL(KB, ¬ B,,)
  if glitter then action ← grab
  else if plan is nonempty then action ← POP(plan)
  else if for some fringe square [i,j], ASK(KB, (¬ P_{i,j} A ¬ W_{i,j})) is true or
          for some fringe square [i,j], ASK(KB, (P_{i,j} ∨ W_{i,j})) is false then do
      plan ← A*-GRAPH-SEARCH(ROUTE-PROBLEM([x,y], orientation, [i,j], visited))
      action ← POP(plan)
  else action ← a randomly chosen move
  return action
```

Figure 7.19 A wumpus-world agent that uses propositional logic to identify pits, wumpuses, and safe squares. The subroutine ROUTE-PROBLEM constructs a search problem whose solution is a sequence of actions leading from [x y] to [i,j] and passing through only previously visited squares.

sentences such as TW~,~ V TW~,~. For a 4 x 4 world, then, we begin with a total of 155 sentences containing 64 distinct symbols.

The agent program, shown in Figure 7.19, TELLS its knowledge base about each new breeze and stench percept. (It also updates some ordinary program variables to keep track of where it is and where it has been-more on this later.) Then, the program chooses where to look next among the fringe squares-that is, the squares adjacent to those already visited. A fringe square [i,j] is provably safe if the sentence (1Pij A lWi j) is entailed by the knowl- edge base. The next best thing is a possibly safe square, for which the agent cannot prove that (Pilj V Wi there is a pit or a wumpus-that is, for which j) is not entailed.

The entailment computation in ASK can be implemented using any of the methods described earlier in the chapter. TT-ENTAILS? (Figure 7.10) is obviously impractical, since it would have to enumerate 264 rows. DPLL (Figure 7.16) performs the required inferences in a few milliseconds, thanks mainly to the unit propagation heuristic. WALKSAT can also be used, with the usual caveats 0about incompleteness.

In wumpus worlds, failures to find a model, given 10,000 flips, invariably correspond to unsatisfiability, so no errors are likely due to incompleteness.

PL-WUMPUS-AGENT works quite well in a small wumpus world. There is, however, something deeply unsatisfying about the agent's knowledge base. KB contains "physics" sentences of the form given in Equations (7.1) and (7.2) for every single square. The larger the environment, the larger the initial knowledge base needs to be. We would much prefer to have just two sentences that say how breezes and stenches arise in all squares. These are

beyond the powers of propositional logic to express. In the next chapter, we will see a more expressive logical language in which such sentences are easy to express.

22.Write a note on circuit based agent.

Circuit-based agents

A circuit-based agent is a particular kind of reflex agent with state.

The percepts are inputs to a sequential circuit-a network of gates, each of which impliments a logical connective, and registers, each of which stores the truth value of a single proposition.

The outputs of the circuit are registers corresponding to actions-for example,

The observant reader will have noticed that this allowed us to finesse the connection between the raw percepts

such as Breeze and the location-specific propositions such as BlI1.

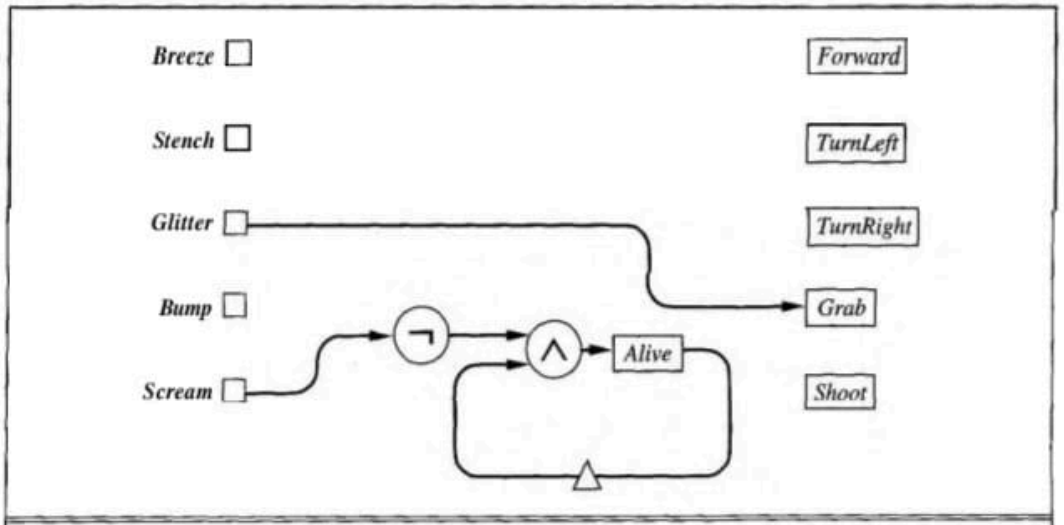


Figure 7.20 Part of a circuit-based agent for the wumpus world, showing inputs, outputs, the circuit for grabbing the gold, and the circuit for determining whether the wumpus is alive. Registers are shown as rectangles and one-step delays are shown as small triangles.

the Grab output is set to true if the agent wants to grab something. If the Glitter input is connected directly to the Grab output, the agent will grab the goal whenever it sees it. (See Figure 7.20.)

Circuits are evaluated in a dataflow fashion: at each time step, the inputs are set and the signals propagate through the circuit. Whenever a gate has all its inputs, it produces an output. This process is closely related to the process of forward chaining in an AND-OR graph such as Figure 7.15(b).

We said in the preceding section that circuit-based agents handle time more satisfactorily than propositional inference-based agents. This is because the value stored in each register gives the truth value of the corresponding proposition symbol at the current time t, rather than having a different copy for each different time step. For example, we might have an Alive register that should contain true when the wumpus is alive and false when it is dead.

This register corresponds to the proposition symbol $Alive^t$, so on each time step it refers to a different proposition. The internal state of the agent-i.e., its memory-is maintained by connecting the output of a register back into the circuit through a delay line. This delivers the value of the register at the previous time step. Figure 7.20 shows an example. The value for Alive is given by the conjunction of the negation of Scream and the delayed value of Alive itself. In terms of propositions, the circuit for Alive implements the biconditional

$$Alive^t \Leftrightarrow \neg Scream^t \wedge Alive^{t-1} .$$

which says that the wumpus is alive at time t if and only if there was no scream perceived at time t (from a scream at t - 1) and it was alive at t - I. We assume that the circuit is initialised with Alive set to true. Therefore, Alive will remain true until there is a scream, whereupon it will become false and stay false. This is exactly what we want.

23.Explain BNF (Backus-Naur Form) grammar of sentences in FOL logic along with operator precedence.

Backus-Naur Form (BNF) is a notation used to formally describe the syntax of languages, including programming languages and formal logics like First-Order Logic (FOL). Here, I'll explain how BNF can be used to describe sentences in First-Order Logic (FOL) and discuss operator precedence within FOL.

BNF Grammar for Sentences in FOL Logic

First-Order Logic consists of terms, formulas, and sentences structured hierarchically. Let's break down the BNF notation for sentences in FOL:

1.*Terms*: Terms are basic elements in FOL that represent objects or entities.

<term> ::= <constant> | <variable> | <function>(<term>, <term>, ...)

- <constant>: Constants represent specific objects (e.g., constants like numbers, names).
- <variable>: Variables represent unspecified objects (e.g., x, y, z).
- <function>: Functions take terms as arguments and represent operations or relations. They can be n-ary, meaning they take a fixed number of arguments.

2. *Formulas*: Formulas in FOL include atomic formulas (predicates applied to terms) and complex formulas built using logical connectives.

<atomic_formula> ::= <predicate>(<term>, <term>, ...)

<complex_formula> ::= <atomic_formula>
| ¬<complex_formula> (Negation)
| <complex_formula> ∧ <complex_formula> (Conjunction)
| <complex_formula> ∨ <complex_formula> (Disjunction)
| ∀ <variable> <complex_formula> (Universal quantification)
| ∃ <variable> <complex_formula> (Existential quantification)

- <predicate>: Predicates are relations or properties that take terms as arguments.

3. *Sentences*: Sentences in FOL are well-formed formulas that do not contain free variables (variables not bound by quantifiers).

<sentence> ::= <complex_formula>

Operator Precedence in FOL

Operator precedence determines the order in which operations (like logical connectives and quantifiers) are applied in expressions. In FOL, the typical precedence hierarchy is:

- *Quantifiers (Highest precedence)*:
  - Universal quantifier ($\forall$)
  - Existential quantifier ($\exists$)

- *Logical Connectives (Lower precedence)*:
  - Negation ($\neg$)
  - Conjunction ($\wedge$)
  - Disjunction ($\vee$)

Operators at the same level of precedence are typically evaluated left to right unless overridden by parentheses. For example, in the formula $\forall x\,(P(x) \wedge Q(x)) \vee R(x)$, conjunction ($\wedge$) is evaluated before disjunction ($\vee$), and quantifiers ($\forall x$, $\exists x$) have the highest precedence.

Example: $\forall x\,(P(x) \wedge Q(x)) \vee R(x)$

- *BNF Representation*:

<sentence> ::= $\forall$<variable> <complex_formula> | <complex_formula>

<complex_formula> ::= <atomic_formula>
| <complex_formula> $\wedge$ <complex_formula>
| <complex_formula> $\vee$ <complex_formula>
| $\neg$<complex_formula>

- *Breakdown*:
  - <complex_formula>: $\forall x\,(P(x) \wedge Q(x)) \vee R(x)$
    - $\forall x\,(P(x) \wedge Q(x))$: Universal quantification over x of the conjunction $P(x) \wedge Q(x)$.
    - $\vee R(x)$: Disjunction of the quantified formula and R(x).

In this example, universal quantification ($\forall$) binds tighter than conjunction ($\wedge$), which binds tighter than disjunction ($\vee$), adhering to the operator precedence rules of FOL.

This BNF grammar and operator precedence help in systematically defining and interpreting expressions and formulas within First-Order Logic.

24.Explain model for first order logic with example.

* First-order logic is also known as Predicate logic or First-order predicate logic.
* First-order logic, like natural language has well defined syntax and semantics.
* It assumes the world contains
    - Objects: people, houses, numbers, colors, baseball games,wars
    - Relations: red, round, prime, brother of, bigger than, part of, comes between,
    - Functions: father of, best friend, one more than, plus,....

Properties of FOL

* It has ability to represent facts about some or all of the objects and relations in the universe
* Represent law and rules extracted from real world
* Useful language for Maths, Philosophy and AI
* Represent facts in realistic manner rather than just true / false
* Makes ontological commitment

First-order logic

Where as propositional logic assumes the world contains facts,
first-order logic (like natural language) assumes the world contains,
Objects: people, houses, numbers, colors, baseball games, wars,
Relations: red, round, prime, brother of, bigger than, part of, comes between
Functions: father of, best friend, one more than, plus, (relations in which there is only one value for a given input)

25.Explain two standard quantifiers in FOL with example.

Universal Quantifier

- Universal quantifier is a symbol of logical representation, which specifies that the statement within its range is true for everything or every instance of a particular thing.
- The Universal quantifier is represented by a symbol ∀, which resembles an inverted A.

  ∀ <variable> <sentence>
- In universal quantifier implication "→" is used.
- If x is a variable, then ∀x is read as:
  - **For all x**
  - **For each x**
  - **For every x**
- Example:
  - **All man drink coffee.**
  - **∀x: man(x) → drink (x, coffee).**

  (There are all x where x is a man who drink coffee)

Existential Quantifier

- Existential quantifiers express that the statement within its scope is true for at least one instance of something.
- It is denoted by the logical operator ∃, which resembles as inverted E. When it is used with a predicate variable then it is called as an existential quantifier.
- In Existential quantifier we always use AND or Conjunction symbol (∧).
- If x is a variable, then existential quantifier will be ∃x or ∃(x). And it will be read as:
  - **There exists a 'x.'**
  - **For some 'x.'**
  - **For at least one 'x.'**
- Example:
  - **Some boys are intelligent.**
  - **∃x: boys(x) ∧ intelligent(x)**

  (There are some x where x is a boy who is intelligent)

26.Write a note on nested quantifiers and equality in FOL.

27.List and Explain the steps of knowledge engineering process in FOL.

28.Explain seven step process for knowledge engineering in electronic circuit domain.

The electronic circuits domain

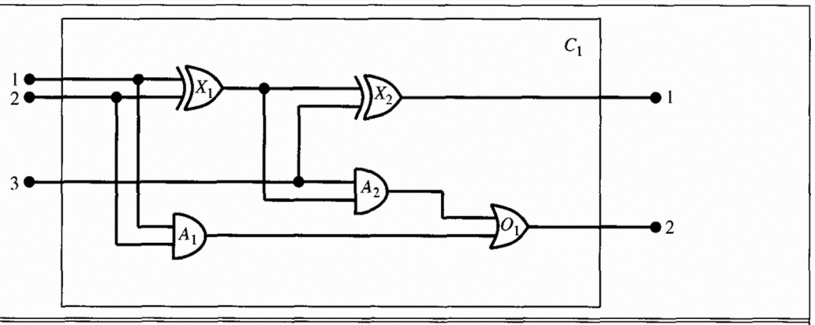


Figure 8.4 A digital circuit C1, purporting to be a one-bit full adder. The first two inputs are the two bits to be added and the third input is a carry bit. The first output is the sum, and the second output is a carry bit for the next adder. The circuit contains two XOR gates, two AND gates and one OR gate.

1.Identify the task

•The knowledge engineer must delineate the range of questions that the knowledge base will support and the kinds of facts that will be available for each specific problem instance.

•For example, does the Wumpus knowledge base need to be able to choose actions or is it required to answer questions only about the contents Finding pits and wumpuses using logical inference

Let us begin with an agent that reasons logically about the location of pits, wumpuses, and

safe squares. It begins with a knowledge base that stat~es the "physics" of the wumpus world.the environment?

• Will the sensor facts include the current location?

•The task will determine what knowledge must be represented in order to connect problem instances to answer .

2.Assemble the relevant knowledge

•The knowledge engineer might already be an expert in the domain, or might need to work with real expert to extract what they know – a process called **Knowledge acquisition.**

•At this stage, the knowledge is not represented formally.

•The idea is to understand the scope of the knowledge base, as determined by the task, and to understand how the domain actually works.

•For the Wumpus world, which is defined by an artificial set of rules, the relevant knowledge is easy to identify.

3.Decide on a vocabulary of predicates, functions and constants.

•That is , translate the important domain-level concepts into logical-level names.

•This involves many questions of knowledge engineering style.

•Like programming style, this can have the significant impact on the eventual success of the project.

•For example, should pits be represented by objects or by a unary predicate on square? Should Wumpus location depend on time?

•Once the choice have been made, the result is a vocabulary that is know as the ontology of the domain

•The word ontology means a particular theory of the nature of being or existence.

•The ontology determines what kinds of things exist, but does not determine their specific properties and interrelationships.

4.Encode general knowledge about the domain

•The knowledge engineer writes down the axioms for all the vocabulary terms.

•This pins down the meaning of the terms, enabling the expert to check the content.

•Often, this step reveals misconceptions or gap in the vocabulary that must be fixed by returning to step3 and iterating through the process.

5.Encode a description of the specific problem instance

•If the ontology is well thought out, this step will be easy.

•It will involve writing simple atomic sentences about instance of concepts that are already part of the ontology.

•For a logical agent, problem instances are supplied by the sensors.

6.Pose queries to the inference procedure and get answers.

•This is where the reward is: we can let the inference procedure operate on the axioms and problem-specific facts to derive the facts we are interested in knowing.

7. Debug the knowledge base

29.Explain inference rules for quantifiers.

Inference rules for quantifiers

Let us begin with universal quantifiers. Suppose our knowledge base contains the standard folkloric axiom stating that all greedy kings are evil: tJ x King(x) A Greedy (x) + Euil(x) . Then it seems quite permissible to infer any of the following sentences: King(John) A Greedy(John) + Evil (John) . King(Richard) A Greedy(Richard) + Evil(Richard) . King(Father(John)) A Greedy (Father(John)~) + Euil(Father(John)) .

Universal Instantiation

The rule of Universal Instantiation (UI for short) salys that we can infer any sentence obtained by substituting a ground term (a term without varialbles) for the variable. ' To write out the inference rule formally, we use the notion of substitutions introduced in Section 8.3. Let SUBST(@, a) denote the result of applying the substitution 8 to the sentence a. Then the rule is written

$$\frac{\forall v \;\; a}{\text{SUBST}(\{v/g\}, \alpha)}$$

for any variable v and ground term g. For example, the three sentences given earlier are obtained with the substitutions 1x1 John), {x/Richard), arid {x/Father(Jolzn)).

Existential Instantiation rule

The corresponding Existential Instantiation rule: for the existential quantifier is slightly more complicated. For any sentence a, variable v, and constant symbol k that does not appear elsewhere in the knowledge base,

$$\frac{\exists v \ \alpha}{\text{SUBST}(\{v/k\}, \alpha)} .$$

For example, from the sentence

3 x Crown(x) A OnHead (x, John)

we can infer the sentence

Crown(C1) A OnHead(Cl, John)

as long as Cl does not appear elsewhere in the knowledge base. Basically, the existential sentence says there is some object satisfying a conditioln, and the instantiation process is just giving a name to that object. Naturally, that name musl. not already belong to another object. Mathematics provides a nice example: suppose we discover that there is a number that is a little bigger than 2.71828 and that satisfies the equation ~d(xy)/dy = xy for x. We can give this number a name, such as e, but it would be a mistake to give it the name of an existing object,

Skolem constant

such as T. In logic, the new name is called a Skolem constant. Existential Instantiation is a special case of a more general process called skolemization, which we cover in Section 9.5. As well as being more complicated than Universal Instantiation, Existential Instantiation plays a slightly different role in inference. Whereas Universal Instantiation can be applied many times to produce many different consequences, Existential Instantiation can be applied once, and then the existentially quantified sentence can be discarded. For example, once we have added the sentence Kill(Murderer, Victim), we no longer need the sentence 3 x Kill(x, Victim). Strictly speaking, the new knowledge base is not logically equivalent

inferentially equivalent

to the old, but it can be shown to be inferentially equivalent in the sense that it is satisfiable exactly when the original knowledge base is satisfiable.

30.Write a note on Unification and Lifting.

Unification

Lifted inference rules require finding substitutions that make different logical expressions

look identical. This process is called **unification** and is a key component of all first-order inference algorithms. The *U NIFY* algorithm takes tcvo sentences and returns a **unifier** for them if one exists:

$$\text{UNIFY}(p, q) = \theta \text{ where } \text{SUBST}(\theta, p) = \text{SUBST}(\theta, q) .$$

Let us look at some examples of how *U NIFY* should behave. Suppose we have a query*Knows (John, x)* : whom does John know? Some answers to this query can be found by find
ing all sentences in the knowledge base that unify with *Knows (John, x).* Here are the results
of unification with four different sentences that might be in the knowledge base.

$$\text{UNIFY}(Knows(John,x), Knows(John,Jane)) = \{x/Jane\}$$
$$\text{UNIFY}(Knows(John,x), Knows(y,Bill)) = \{x/Bill, y/John\}$$
$$\text{UNIFY}(Knows(John,x), Knows(y,Mother(y))) = \{y/John, x/Mother(John)\}$$
$$\text{UNIFY}(Knows(John,x), Knows(x,Elizabeth)) = Jail.$$

The last unification fails because *x* cannot take on the values *John* and *Elizabeth* at the
same time. Now, remember that *Knows(x, Elizabeth)* means "Everyone knows Elizabeth,"
so we *should* be able to infer that John knows Elizabeth. The problem arises only because
the two sentences happen to use the same variable name, 1:. The problem can be avoided
by **standardizing apart** one of the two sentences being unified, which means renaming its
variables to avoid name clashes. For example, we can rename *x* in *Knows(x, Elizabeth)* to
217 (a new variable name) without changing its meaning. Now the unification will work:

$$\text{UNIFY}(Knows(John,x), Knows(z_{17}, Elizabeth)) = \{x/Elizabeth, z_{17}/John\}.$$

There is one more complication: we said that *U NIFY* should return a substitution that makes the two arguments look the same. But there could be more than one such unifier. For example,

$\text{UNIFY}(Knows(John,x), Knows(y,z))$ could return $\{y/John, x/z\}$

or

{y/ John, z/ *John, z/ John).* The first unifier gives *Knows(John, z)* as the result of unification, whereas the second gives *Knows (John, John),* The second result could be obtained from the first by an additional substitution *{z/ John);* we say that the first unifier is *more general* than the second, because it places fewer restrictions on the values of the variables. It turns out that, for every unifiable pair of expressions, there

is a single **most general unifier**(or *MGU)* that is unique up to renaming of variables. In this case it is *{y/ John, x/z).*

An algorithm for computing most general unifiers is shown in Figure 9.1

The process is very simple: recursively explore the two expressions simultaneously "side by side," building up a unifier along the way, but failing if two corresponding points in the structures do not match. There is one expensive step: when matching a variable against a complex term,one must check whether the variable itself occurs inside the term; if it does, the match fails because no consistent unifier can be constructed. This so-called **occur check** makes the complexity of the entire algorithm quadratic in the size of the expressions being unified.

Some systems, including all logic programming systems, simply omit the occur check and

sometimes make unsound inferences as a result; other system:s use more complex algorithms

with linear-time complexity.

```
function UNIFY(x, y, θ) returns a substitution to make x and y identical
  inputs: x, a variable, constant, list, or compound
          y, a variable, constant, list, or compound
          θ, the substitution built up so far (optional, defaults to empty)

  if θ = failure then return failure
  else if x = y then return θ
  else if VARIABLE?(x) then return UNIFY-VAR(x, y, θ)
  else if VARIABLE?(y) then return UNIFY-VAR(y, x, θ)
  else if COMPOUND?(x) and COMPOUND?(y) then
      return UNIFY(ARGS[x], ARGS[y], UNIFY(OP[x], OP[y], θ))
  else if LIST?(x) and LIST?(y) then
      return UNIFY(REST[x], REST[y], UNIFY(FIRST[x], FIRST[y], θ))
  else return failure
```

```
function UNIFY-VAR(var, x, θ) returns a substitution
  inputs: var, a variable
          x, any expression
          θ, the substitution built up so far

  if {var/val} ∈ θ then return UNIFY(val, x, θ)
  else if {x/val} ∈ θ then return UNIFY(var, val, θ)
  else if OCCUR-CHECK?(var, x) then return failure
  else return add {varlx) to θ
```

Figure 9.1 The unification algorithm. The algorithm works by comparing the structures of the inputs, element by element. The substitution θ that is the argument to UNIFY is built up along the way and is used to make sure that later comparisons are consistent with bindings that were established earlier. In a compound expression, such as $F(A, B)$, the function OP picks out the function symbol F and the function ARGS picks out the argument list (A, B).

31.Explain Unification algorithm.

32.Explain forward chaining algorithm in FOL.

Forward chaining is a data-driven reasoning method used in artificial intelligence, particularly in rule-based systems, for inferring new facts from known facts and rules. In first-order logic (FOL), forward chaining uses a set of inference rules to derive conclusions from given premises.

Forward Chaining Algorithm Steps

1. **Initialization**:
   - Start with a set of known facts (initial database).
   - Initialise the working memory with these facts.
2. **Rule Matching**:
   - Identify rules whose premises match the facts in the working memory.
   - In FOL, this involves unifying the premises with the facts.
3. **Rule Application**:
   - Select and apply a matching rule.
   - Infer new facts from the rule's conclusion and add them to the working memory.
4. **Iteration**:
   - Repeat rule matching and application until:
     - A goal fact is derived, or
     - No more rules can be applied.
5. **Termination**:
   - The process stops when the goal fact is derived or no new inferences can be made.

Example Facts:

1. American(West)American(West)American(West)
2. Missile(M1)Missile(M_1)Missile(M1)
3. Owns(Nono,M1)Owns(Nono, M_1)Owns(Nono,M1)
4. Sells(West,M1,Nono)Sells(West, M_1, Nono)Sells(West,M1,Nono)
5. Enemy(Nono,America)Enemy(Nono, America)Enemy(Nono,America)

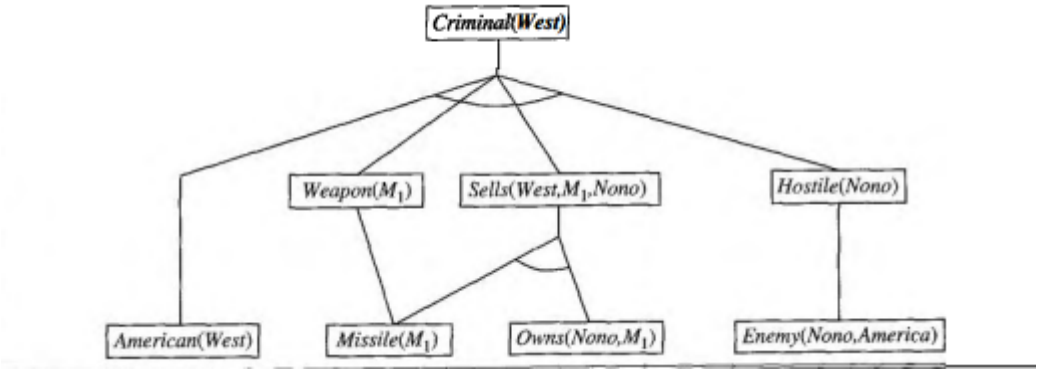
Example Rules:

1. American(x)∧Weapon(y)∧Sells(x,y,z)∧Hostile(z)→Criminal(x)
2. Missile(x)→Weapon(x)
3. Enemy(x,America)→Hostile(x)

Query:

- Is Criminal(West)Criminal(West)Criminal(West) true?

Diagram:



Pseudocode:

```
def forward_chaining(facts, rules, goal):
    working_memory = set(facts)
    while True:
        new_facts = set()
        for rule in rules:
            if rule.premise_matches(working_memory):
                inferred_fact = rule.apply()
                if inferred_fact == goal:
                    return True
                new_facts.add(inferred_fact)
        if not new_facts:
            return False
        working_memory.update(new_facts)
```

Algorithm Steps Applied to Example:

1. **Initialization**:
    - Facts in KB: {American(West),Missile(M1),Owns(Nono,M1),Sells(West,M1,Nono),Enemy(Nono,America)}
2. **First Iteration**:
    - Apply Rule 2: Missile(M1)→Weapon(M1)
    - New fact inferred: Weapon(M1)
3. **Second Iteration**:
    - Apply Rule 3: Enemy(Nono,America)→Hostile(Nono)
    - New fact inferred: Hostile(Nono)
4. **Third Iteration**:
    - Apply Rule 1: American(West)∧Weapon(M1)∧Sells(West,M1,Nono)∧Hostile(Nono)→Criminal(West)

- All premises match known facts.
- New fact inferred: Criminal(West)

Result:

- The query Criminal(West) is true as it has been inferred through forward chaining.

Summary

Forward chaining in FOL involves initialising with known facts, matching rules with these facts, applying rules to infer new facts, and iterating this process until the query is confirmed or no new facts are generated. In the given example, forward chaining successfully infers Criminal(West) based on the initial facts and rules in the knowledge base.

33.Explain backward chaining algorithm in FOL.

Backward chaining in First-Order Logic (FOL), also known as backward reasoning, is a method used for automated theorem proving and reasoning in artificial intelligence and logic programming.

Figure 9.6 shows a simple backward-chaining algorithm, FOL-BC-ASK. It is called with a list of goals containing a single element, the original query, and returns the set of all substitutions
satisfying the query. The list of goals can be thought of as a "stack" waiting to be .worked on; if all of them can be satisfied, then the current branch of the proof succeeds. The algorithm takes the first goal in the list and finds every clause in the knowledge base whose positive literal, or head, unifies with the goal. Each such clause creates a new recursive call in which the premise, or body, of the clause is added to the goal stack. Remember that facts are clauses with a head but no body, so when a goal unifies with a known fact, no new subgoals
are added to the stack and the goal is solved. Figure 9.7 is the proof tree for deriving Criminal(West) from sentences (9.3) through (9.10).

```
function FOL-BC-ASK(KB, goals, θ) returns a set of substitutions
  inputs: KB, a knowledge base
          goals, a list of conjuncts forming a query (θ already applied)
          θ, the current substitution, initially the empty substitution { }
  local variables: answers, a set of substitutions, initially empty

  if goals is empty then return {θ}
  q' ← SUBST(θ, FIRST(goals))
  for each sentence r in KB where STANDARDIZE-APART(r) = (p1 ∧ ... ∧ pn ⇒ q)
        and θ' ← UNIFY(q, q') succeeds
     new-goals ← [p1, ..., pn | REST(goals)]
     answers ← FOL-BC-ASK(KB, new-goals, COMPOSE(θ', θ)) ∪ answers
  return answers
```

Figure 9.6 A simple backward-chaining algorithm.

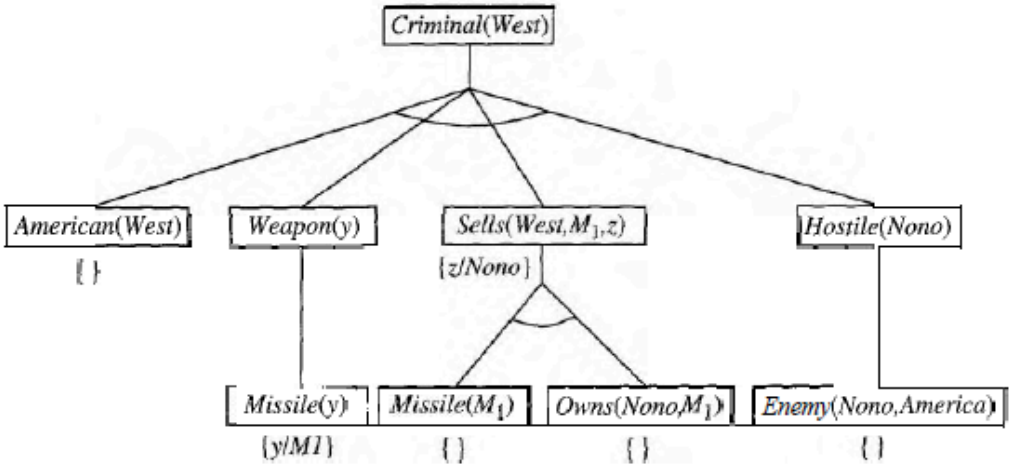


Figure 9.7 Proof tree constructed by backward chaining to prove that West is a criminal. The tree should be read depth first, left to right. To prove *Criminal(West)*, we have to prove the four conjuncts below it. Some of these are in the knowledge base, and others require further backward chaining. Bindings for each successful unification are shown next to the corresponding subgoal. Note that once one subgoal in a conjunction succeeds, its substitution is applied to subsequent subgoals. Thus, by the time *FOL-BC-ASK* gets to the last conjunct, originally *Hostile*(*z*), *z* is already bound to *Nono*.

The algorithm uses **composition** of substitutions. COMPOSE(θ_1, θ_2) is the substitution whose effect is identical to the effect of applying each substitution in turn. That is,

$$\text{SUBST}(\text{COMPOSE}(\theta_1, \theta_2), p) = \text{SUBST}(\theta_2, \text{SUBST}(\theta_1, p)) .$$

In the algorithm, the current variable bindings, which are stored in 8, are composed with the bindings resulting from unifying the goal with the clause head, giving a new set of current

bindings for the recursive call.

Example of backward chaining

- The information provided in the previous example (forward chaining) can be used to provide a simple explanation of backward chaining. Backward chaining can be explained in the following sequence.
- B
- A->B
- A
- B is the goal or endpoint, that is used as the starting point for backward tracking. A is the initial state. A->B is a fact that must be asserted to arrive at the endpoint B.
- A practical example of backward chaining will go as follows:
- Tom is sweating (B).
- If a person is running, he will sweat (A->B).
- Tom is running (A).